

HYPERPARAMETER SENSITIVITY ANALYSIS

NPS Grievance Classification | DistilBERT Multi-Task Learning | April 2026

1. OBJECTIVE AND APPROACH

Following the completion of Experiment 2 (Model 2) and Experiment 3 (Model 3), a systematic sensitivity analysis was conducted to empirically validate the optimal hyperparameter configuration for the next full-scale training run. Rather than relying on assumptions or literature defaults, each parameter was isolated and tested independently — holding all other parameters fixed at their best-known values — to quantify its precise impact on model performance.

Why this matters: Model 3 training revealed that priority classification performance was significantly affected by the class weight configuration. Rather than guessing corrections, this analysis provides empirical evidence for every configuration decision, making the next training run defensible and reproducible.

Experimental Setup

Parameter	Value
Base model	distilbert-base-uncased, initialized from Model 2 weights
Sensitivity subset	1,00,000 rows (10% stratified sample from full train set)
Epochs per run	1 (sufficient for relative comparison — keeps compute cost low)
Validation set	2,16,962 rows (full validation set — consistent across all runs)
Best model criterion	P1 recall ≥ 0.60 AND highest average macro F1 across all 3 heads
Experiment tracking	MLflow — experiment NPS_Sensitivity_Analysis_V1
Total combinations trained	22 (across 4 phases, with 4 results reused across phases)

Selection criterion rationale: P1 recall ≥ 0.60 is enforced as a hard constraint because Priority 1 represents 74.9% of all grievances — a model that fails to identify the majority class reliably is operationally unusable regardless of macro F1. Within combinations that satisfy this constraint, average macro F1 is maximized because macro F1 treats all classes equally and penalizes poor rare-class performance.

2. PHASE 1 — PRIORITY CLASS WEIGHT OPTIMIZATION

Motivation

Model 2 used sklearn auto-balanced class weights for Priority, which resulted in Priority 1 weighted F1 of 64.56% with poor P2 and P3 recall (47% and 53% respectively). The hypothesis was that manually increasing P2 and P3 weights could improve minority class recovery. Phase 1 tested whether manual priority weight adjustment outperforms auto-balanced weights.

Combinations Tested

Combination	P1 Weight	P2 Weight	P3 Weight	Hypothesis
combo_1_pure_auto	0.447 (auto)	2.382 (auto)	2.898 (auto)	Model 2 baseline — pure sklearn balanced
combo_2_auto_p1_moderate_p2p3	0.447	3.5	4.0	Auto P1, moderate manual push on P2/P3
combo_3_auto_p1_strong_p2p3	0.447	4.0	5.0	Auto P1, strong push on P2/P3
combo_4_higher_p1_moderate_p2p3	0.600	3.5	4.0	Slightly higher P1 with moderate P2/P3
combo_5_higher_p1_strong_p2p3	0.600	4.0	5.0	Slightly higher P1 with strong P2/P3

Results

Combination	P1 Recall	P2 Recall	P3 Recall	P mF1	C mF1	SC mF1	Avg mF1	Overall Acc
combo_1_pure_auto	0.6796	0.3722	0.5359	0.4828	0.4441	0.2109	0.3793 ✓	0.5262
combo_2_auto_p1_moderate	0.4357	0.5445	0.5549	0.4120	0.4411	0.2121	0.3551	0.4755
combo_3_auto_p1_strong	0.3531	0.5088	0.6422	0.3736	0.4401	0.2075	0.3404	0.4543
combo_4_higher_p1_moderate	0.6199	0.3941	0.5703	0.4654	0.4443	0.2083	0.3727	0.5123
combo_5_higher_p1_strong	0.5301	0.4296	0.6114	0.4387	0.4419	0.2136	0.3647	0.4959

Winner: combo_1_pure_auto — sklearn auto-balanced weights [0.447, 2.382, 2.898]

Avg mF1: 0.3793 | P1 Recall: 0.6796 | Overall Accuracy: 52.62%

Finding

Every manual weight combination that pushed P2 and P3 higher caused P1 recall to drop below the 0.60 threshold. The sklearn auto-balanced weights provide the best trade-off — the model maintains operationally acceptable P1 recall while making meaningful predictions for minority

classes. Manual weight injection beyond auto-balanced values introduces a zero-sum trade-off rather than an improvement.

3. PHASE 2 — TASK LOSS WEIGHT OPTIMIZATION

Motivation

Task weights determine the relative contribution of each classification head to the combined training loss. The assumption carried from Experiments 2 and 3 was Priority×1.5, Category×1.2, Sub-Category×1.0. Phase 2 tested whether this assumption is empirically optimal or whether alternative task weight configurations produce better overall performance.

Combinations Tested

Combination	Task Weights [P, C, SC]	Hypothesis
tw_1_equal	[1.0, 1.0, 1.0]	No task bias — tests if current weighting actually helps
tw_2_current_assumption	[1.5, 1.2, 1.0]	Current configuration — reused from Phase 1, no retraining
tw_3_priority_only_focus	[2.0, 1.0, 1.0]	Priority 2x — tests pure priority focus
tw_4_priority_category_boost	[2.0, 1.5, 1.0]	Priority + Category boosted — balanced improvement test
tw_5_subcategory_focus	[1.0, 1.0, 2.0]	Sub-category 2x — tests if SC boost recovers rare classes
tw_6_category_focus	[1.0, 2.0, 1.0]	Category 2x — tests if category boost helps weak categories

Results

Combination	Task Weights	P1 Recall	P2 Recall	P mF1	C mF1	SC mF1	Avg mF1	Overall Acc
tw_1_equal	[1.0, 1.0, 1.0]	0.6744	0.3698	0.4811	0.4401	0.2117	0.3776	0.5255
tw_2_current_assumption	[1.5, 1.2, 1.0]	0.6796	0.3722	0.4828	0.4441	0.2109	0.3793 ✓	0.5262
tw_3_priority_only_focus	[2.0, 1.0, 1.0]	0.6712	0.3492	0.4783	0.4411	0.2125	0.3773	0.5254
tw_4_priority_category_boost	[2.0, 1.5, 1.0]	0.6570	0.3644	0.4760	0.4406	0.2126	0.3764	0.5204
tw_5_subcategory_focus	[1.0, 1.0, 2.0]	0.6687	0.3478	0.4767	0.4363	0.2089	0.3740	0.5203

Combination	Task Weights	P1 Recall	P2 Recall	P mF1	C mF1	SC mF1	Avg mF1	Overall Acc
tw_6_category_focu s	[1.0, 2.0, 1.0]	0.6662	0.3625	0.4773	0.4440	0.2138	0.3784	0.5246

Winner: tw_2_current_assumption — Task Weights [1.5, 1.2, 1.0]

Avg mF1: 0.3793 | P1 Recall: 0.6796 | Overall Accuracy: 52.62%

Finding

The differences across all task weight combinations are small — avg mF1 ranges only 0.0053 across all 6 combinations. This indicates that task weights are a low-sensitivity parameter for this model at this scale. The original [1.5, 1.2, 1.0] configuration is marginally optimal, confirming that prioritizing the Priority head during training is the correct inductive bias. Boosting sub-category weight did not recover rare classes — rare class performance is driven by data volume and class weights, not task weights.

4. PHASE 3 — LABEL SMOOTHING OPTIMIZATION

Motivation

Label smoothing prevents overconfident predictions by replacing hard targets (0 or 1) with soft targets. For imbalanced datasets, the hypothesis is that smoothing can improve calibration on minority classes by reducing the model's certainty on dominant classes. Experiments 2 and 3 used label_smoothing=0.05 based on an initial assumption. Phase 3 tested the full practical range from 0.0 to 0.2 to identify the empirically optimal value.

Combinations Tested

Combination	Label Smoothing	Hypothesis
ls_1_no_smoothing	0.0	Hard targets — tests if any smoothing is harmful on this dataset
ls_2_minimal_0_025	0.025	Minimal smoothing — fills gap between 0.0 and baseline 0.05
ls_3_current_0_05	0.05	Current baseline — reused from Phase 1, no retraining
ls_4_standard_0_1	0.1	Standard NLP value — most common BERT fine-tuning recommendation
ls_5_stronger_0_15	0.15	Stronger smoothing — tests if more regularization helps rare classes
ls_6_upper_bound_0_2	0.2	Upper bound — targets too flat, expected degradation

Results

Combination	Label Smoothing	P1 Recall	P2 Recall	P mF1	C mF1	SC mF1	Avg mF1	Overall Acc
ls_1_no_smoothing	0.0	0.7116	0.3306	0.4893	0.5128	0.2573	0.4198 ✓	0.5455
ls_2_minimal_0_025	0.025	0.6839	0.3556	0.4827	0.4796	0.2361	0.3995	0.5364
ls_3_current_0_05	0.05	0.6796	0.3722	0.4828	0.4441	0.2109	0.3793	0.5262
ls_4_standard_0_1	0.1	0.6008	0.4028	0.4589	0.4130	0.1863	0.3527	0.4727
ls_5_stronger_0_15	0.15	0.4746	0.4429	0.4168	0.3559	0.1644	0.3124	0.3526
ls_6_upper_bound_0_2	0.2	0.3841	0.4519	0.3797	0.3167	0.1519	0.2828	0.2782

Winner: ls_1_no_smoothing — Label Smoothing = 0.0

Avg mF1: 0.4198 | P1 Recall: 0.7116 | Overall Accuracy: 54.55% | Significant jump from Phase 2 baseline

Finding

Label smoothing shows a clear and monotonic negative relationship with performance on this dataset. Every increment of smoothing reduces avg mF1 and overall accuracy. At 0.2, category accuracy collapses to 21.7% — the targets become too flat for the model to learn meaningful class boundaries.

The counter-intuitive result — that no smoothing is optimal — is explained by the class weight mechanism already in place. Class weights applied inside CrossEntropyLoss already provide calibration pressure on minority classes. Adding label smoothing on top of class weights over-regularizes the loss signal, diluting the class weight effect. The two mechanisms are partially redundant, and label smoothing creates net harm in this configuration.

5. PHASE 4 — LEARNING RATE OPTIMIZATION (ENCODER AND HEAD)

Phase 4 was split into two sequential steps. Step 1 optimized the encoder learning rate with the head learning rate fixed. Step 2 then optimized the head learning rate with the winning encoder LR fixed. This sequential approach isolates each variable and avoids a combinatorial explosion of joint LR testing.

Phase 4 Step 1 — Encoder Learning Rate

Motivation: DistilBERT's encoder contains 66M pre-trained parameters that have already learned rich language representations. When fine-tuning on a domain-specific dataset, the encoder learning rate controls how aggressively these representations are adapted to NPS grievance language. Too

low — the encoder under-adapts and fails to capture domain-specific patterns. Too high — the encoder overwrites pre-trained knowledge (catastrophic forgetting). The optimal encoder LR is the point where NPS-specific adaptation is maximized without damaging pre-trained representations.

Why encoder LR requires separate tuning from head LR: The classification heads are task-specific layers initialized from Model 2 weights — they have already been partially adapted to this problem. The encoder, however, needs to be tuned at a different rate because it balances pre-trained knowledge preservation against domain adaptation. Using a single LR for both, as in Experiments 1 and 2, conflates two fundamentally different learning objectives.

Combination	Encoder LR	P1 Recall	P2 Recall	P mF1	C mF1	SC mF1	Avg mF1	Overall Acc
elr_1_very_conservative	1e-6	0.7041	0.3475	0.4885	0.5126	0.2598	0.4203	0.5432
elr_2_conservative	3e-6	0.7005	0.3506	0.4880	0.5184	0.2677	0.4247	0.5444
elr_3_current	5e-6	0.7116	0.3306	0.4893	0.5128	0.2573	0.4198	0.5455
elr_4_moderate	1e-5	0.7359	0.3327	0.4955	0.5183	0.2731	0.4290 ✓	0.5558
elr_5_aggressive	3e-5	0.6785	0.3791	0.4818	0.5030	0.2494	0.4114	0.5346
elr_6_upper_bound	5e-5	0.7319	0.3316	0.4897	0.4978	0.2420	0.4098	0.5454

Winner: elr_4_moderate — Encoder LR = 1e-5

Avg mF1: 0.4290 | P1 Recall: 0.7359 | Overall Accuracy: 55.58%

Performance peaks at 1e-5 and degrades on both sides. Too conservative (1e-6) results in under-adaptation — the model cannot fully learn NPS-specific grievance language patterns. Too aggressive (3e-5, 5e-5) shows performance decline as the encoder begins to overwrite pre-trained semantic representations. The encoder LR is a high-sensitivity parameter — the avg mF1 spread across combinations (0.4098–0.4290) is 4x larger than the task weight spread from Phase 2.

Phase 4 Step 2 — Head Learning Rate

Motivation: With the encoder LR fixed at 1e-5, Phase 4 Step 2 tested whether the classification heads benefit from a higher learning rate than the encoder. The standard recommendation in transformer fine-tuning literature is a 5–10x higher head LR, under the assumption that task-specific heads need faster adaptation than the pre-trained encoder. However, since all three classification heads are initialized from Model 2 weights rather than random initialization, this assumption may not hold.

Combination	Encoder LR	Head LR	Ratio	P1 Recall	P mF1	C mF1	SC mF1	Avg mF1	Overall Acc
h1r_1_current	1e-5	1e-5	1:1	0.7359	0.4955	0.5183	0.2731	0.4290 ✓	0.5558

Combination	Encoder LR	Head LR	Ratio	P1 Recall	P mF1	C mF1	SC mF1	Avg mF1	Overall Acc
h1r_2_mild	1e-5	5e-5	5x	0.6950	0.4859	0.5203	0.2608	0.4223	0.5451
h1r_3_standard	1e-5	1e-4	10x	0.7242	0.4918	0.5123	0.2525	0.4189	0.5441
h1r_4_aggressive	1e-5	5e-4	50x	0.6907	0.4836	0.5116	0.2399	0.4117	0.5349
h1r_5_very_aggressive	1e-5	1e-3	100x	0.6644	0.4644	0.5045	0.2316	0.4002	0.5267
h1r_6_upper_bound	1e-5	2e-3	200x	0.6917	0.4784	0.5032	0.2123	0.3980	0.5236

Winner: h1r_1_current — Head LR = 1e-5 (1:1 ratio with encoder LR)

Avg mF1: 0.4290 | P1 Recall: 0.7359 | Overall Accuracy: 55.58%

Increasing head LR beyond encoder LR consistently degrades performance across all tested ratios. This directly contradicts the standard literature recommendation, and the reason is clear: the classification heads are

initialized from Model 2 weights — they are not randomly initialized. Applying a higher LR to already-adapted heads causes unnecessary weight disruption. The 1:1 ratio allows both the encoder and heads to adapt at the same controlled pace, which is optimal when the starting point is an already-trained model rather than random initialization.

6. CONSOLIDATED RESULTS AND VALIDATED CONFIGURATION

Phase	Parameter Tuned	Values Tested	Winner	Avg mF1 (Winner)	Sensitivity
Phase 1	Priority class weights	5 combinations	Auto-balanced [0.447, 2.382, 2.898]	0.3793	High — P1 recall constraint is binding
Phase 2	Task loss weights [P, C, SC]	6 combinations	[1.5, 1.2, 1.0]	0.3793	Low — mF1 range = 0.0053
Phase 3	Label smoothing	6 combinations	0.0 (no smoothing)	0.4198	High — monotonic degradation with smoothing
Phase 4 Step 1	Encoder learning rate	6 combinations	1e-5	0.4290	High — mF1 range = 0.0192
Phase 4 Step 2	Head learning rate	6 combinations	1e-5 (1:1 with encoder)	0.4290	Moderate — equal LR optimal for warm-heads

Validated Configuration for Next Training Run

Parameter	Validated Value	Source
Priority class weights	Auto-balanced (sklearn balanced)	Phase 1
Task loss weights	[1.5, 1.2, 1.0] — Priority, Category, Sub-Category	Phase 2
Label smoothing	0.0	Phase 3
Encoder learning rate	1e-5	Phase 4 Step 1
Head learning rate	1e-5 (equal to encoder LR)	Phase 4 Step 2

7. CONCLUSION AND NEXT STEPS

What this analysis confirms:

- **Auto-balanced class weights are optimal.** Manual weight injection introduces a zero-sum trade-off between P1 recall and P2/P3 recall. Sklearn balanced weights are the correct configuration.
- **Task weights [1.5, 1.2, 1.0] are empirically validated.** The original assumption is confirmed — priority-first task weighting is the correct inductive bias, though the sensitivity is low.
- **Label smoothing is harmful on this dataset.** The class weight mechanism already provides the calibration effect that label smoothing would normally add. Using both simultaneously over-regularizes the loss.
- **Encoder LR = 1e-5 is the critical finding.** This is 2x higher than the 5e-6 used in Experiments 2 and 3, and produces a measurable improvement in avg mF1 (+0.009 on the 1-epoch sensitivity subset).
- **Equal encoder and head LR is correct for warm-start fine-tuning.** Standard literature recommendations for higher head LR assume random head initialization. When heads are loaded from a pre-trained checkpoint, equal LR's are optimal.

Remaining Phases to Complete

The following parameters have not yet been tuned and will be addressed in subsequent sensitivity phases before finalizing the next full-scale training run:

Planned Phase	Parameter	Rationale
Phase 5	Dropout rate	Model 3 used dropout=0.5 as an assumption — optimal value needs empirical validation
Phase 6	Warmup ratio	Current warmup of 10% may be suboptimal — too short risks instability, too long slows convergence
Phase 7	Gradient clip threshold	Clip=0.5 in Model 3 was conservative — optimal threshold depends on the validated LR configuration

Planned Phase	Parameter	Rationale
Phase 8	Max class weight cap	Cap=20.0 controls the maximum penalty for rare classes — interaction with auto-balanced weights needs validation
Phase 9	Combined config validation	Final run with all validated parameters combined to confirm there are no interaction effects between phases

Each remaining phase will follow the same rigorous protocol: 1-epoch runs on the 1L subset, same validation set, same MLflow tracking, same P1 recall constraint. Results from all phases will be combined into the final training configuration before launching the next full-scale model training.